DAY 5 OF THE AI ENGINEERING INTERVIEW SERIES

40 Embeddings & Vector DB Questions

From Cosine Similarity to HNSW, ANN, and hybrid search.

What actually happens between: User Query → Vector DB → Context → LLM

VECTOR CONCEPTS COVERED:

HNSW Graph Indexes — IVF Clustering — Dimension & Memory Bounds — Cosine vs Euclidean — Hybrid BM25 Fusion — Reranking

40 Q&As — Visual Pipelines — Cheat Sheet →

Devadharshani A

Follow Topmate Repost

0.92 Similarity — but the answer is WRONG

INTERVIEWER EXPECTS: Differentiating semantic vector similarity from text relevance and factual accuracy.

Similarity only indicates geometric vector alignment. The retrieved chunk can still be:

- Semantically related but fails to contain the target answer.
- From an outdated document version or missing metadata flags.



INTERVIEW TIP

Similarity ≠ relevance ≠ correctness. A production RAG system requires separate retrieval evaluation metrics, not just database scores.

Why can't we compare against 10M vectors?

INTERVIEWER EXPECTS: Exhaustive search time complexity (O(N * d)) and limits of brute force computation.

Exhaustive comparisons require distance evaluations across every entry. For dimension d and dataset size N, search cost is **O(N × d)**:



INTERVIEW TIP

ANN (Approximate Nearest Neighbor) indexes trade exactness for latency gains and query-per-second throughput increases.

Search is 3× faster — but accuracy dropped

INTERVIEWER EXPECTS: Latency vs. Recall trade-off knowledge and ANN configuration parameter tuning.

Reducing exploration bounds (e.g. lowering HNSW `efSearch` or IVF `nprobe`) speeds up queries but excludes correct candidates:



INTERVIEW TIP

Never optimize search latency without monitoring search recall metrics. Use ground-truth exact index queries to measure recall degradation.

What is an embedding?

INTERVIEWER EXPECTS: Going beyond "numbers for words" — continuous representation spaces.

An embedding is a learned numerical representation mapping input data into a vector space where semantic proximity is measured geometrically:



INTERVIEW TIP

Describe embeddings as maps: "Embeddings place semantic meaning on geometric axes where orientation and distance represent conceptual relationships."

Why use embeddings for semantic search?

INTERVIEWER EXPECTS: Distinguishing semantic vector search from keyword search.

Keyword search requires lexical matches. Embedding search maps distinct words with similar meanings to close vector coordinates:

- **Keyword query:**"reset passcode" vs "change credentials" (No keyword overlap).
- **Embedding query:**Maps both terms close together in semantic space.

INTERVIEW TIP

Vector search parses query intent, bypassing strict vocabulary overlap constraints of keyword indexes.

What does dimension mean?

INTERVIEWER EXPECTS: Practical understanding of memory footprints and computation footprints.

Dimension defines the length of the vector float array. Larger dimensions increase memory consumption and distance computation workloads:

1,000,000 vectors × 1,536 dimensions

```
1M × 1536 × 4 bytes ≈ 6.14 GB
```

INTERVIEW TIP

A larger dimension size is not a retrieval performance guarantee — design indices based on vector dimension counts and hardware resources.

Are larger dimensions always better?

INTERVIEWER EXPECTS: Dismissing "bigger is always better" — evaluating model training quality.

No. A vector space's representation capacity is dictated by model training datasets and alignment constraints, not raw size parameters:

- ✓ A well-aligned 768-dimension vector can outperform a noisy 1536-dimension output.
- ✓ Benchmarking on target domain data is required to justify memory costs.

INTERVIEW TIP

"I would benchmark search recall across multiple candidate models on domain data instead of assuming larger vector sizes translate to quality search."

What is cosine similarity?

INTERVIEWER EXPECTS: Conceptual understanding of angular orientation similarity formulas.

Cosine similarity measures the angle between vectors, ignoring raw segment length differences:



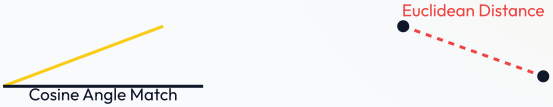
INTERVIEW TIP

Cosine similarity tracks vector orientation rather than magnitude. It is ideal when document length variance scale vectors outward.

Cosine vs Euclidean distance

INTERVIEWER EXPECTS: Distinguishing directional alignment from physical geometric distance.

Cosine checks the angle between vector coordinates. Euclidean distance calculates the physical distance between points:



IMPORTANT NUANCE

If vectors are normalized to unit length, Euclidean distance and Cosine similarity rank items identically: $||A - B||^2 = 2 - 2\cos(A,B)$.

Dot product vs cosine similarity

INTERVIEWER EXPECTS: Impact of vector length on similarity scores.

Dot product measures vector alignment and magnitude. Unlike cosine, dot product scores scale with vector lengths:



INTERVIEW TIP

If vectors are normalized, dot product and cosine ranking behave identically. Choose dot product for normalized vectors to speed up queries.

Why use cosine similarity for text?

INTERVIEWER EXPECTS: Impact of document length on vector magnitude coordinates.

Varying text lengths scale vector magnitudes outward. Cosine similarity isolates directional alignment, ignoring length-scaling offsets:

- ✓ A short document and its duplicate-extended version point in the same direction.
- ✓ Cosine evaluates them as similar, whereas Euclidean scores them as distant.

IMPORTANT NUANCE

Always align your search metric with the model training objective — models optimized with inner product require normalized vectors.

What is nearest-neighbor search?

INTERVIEWER EXPECTS: Definition of kNN retrieval pipelines.

Nearest-neighbor (kNN) search parses database vectors to identify the most similar entries according to chosen similarity metrics:



INTERVIEW TIP

Nearest neighbor search represents the core retrieval engine of RAG applications, converting similarities to prompt context.

What is ANN search?

INTERVIEWER EXPECTS: Explaining sub-linear search complexity benefits.

Approximate Nearest Neighbor (ANN) search evaluates candidate sub-regions rather than executing full database comparisons:

- **Work reduction:** Cuts computation boundaries down to candidate zones.
- **Trade-off:** Deliberately trades a fractional percentage of search recall for latency speedups.

INTERVIEW TIP

ANN converts linear search complexity O(N) into sub-linear log(N) patterns, enabling massive database scalability.

Exact vs ANN search

INTERVIEWER EXPECTS: Trade-offs across scale, speed, and recall.

Choosing search indexing modes depends on resource budgets and throughput metrics:

Search Mode	Recall	Latency	Memory
Exact (Flat)	100%	Slow	Low
ANN (HNSW/IVF)	~95-99%	Fast	High

INTERVIEW TIP

Exact search serves as your baseline ground truth. Use it to measure recall gaps introduced by ANN indexing configurations.

What is HNSW?

INTERVIEWER EXPECTS: Hierarchical graph search structures.

HNSW (Hierarchical Navigable Small World) structures data into multi-layer navigation graphs, connecting nearby vectors:



INTERVIEW TIP

HNSW graphs provide fast retrieval speeds and high search recall, but they require a significant memory footprint to hold graph links.

How does HNSW search work?

INTERVIEWER EXPECTS: Multi-layer search traversal routing logic.

Queries enter at the top layers to traverse wide distances, then descend to dense lower layers to locate local nearest neighbors:

- **Upper Layers:**High-speed, coarse routing.
- **Lower Layers:**Fine-grained, local search alignment.

SYSTEM CONCEPT

This skips evaluating out-of-bounds nodes, reducing computation down to log(N) traversal operations.

What does M mean in HNSW?

INTERVIEWER EXPECTS: Explaining max link connectivity constraints per node.

M defines the maximum number of bidirectional graph links established per vector node during indexing:



M = 4 links

TUNING AXIOM

Higher M values improve graph recall and connectivity, but increase index building times and memory footprints.

What is efSearch in HNSW?

INTERVIEWER EXPECTS: Query-time search candidate pool exploration control.

efSearch controls the size of the dynamic candidate list evaluated during query-time graph traversals:

- ✓ **Low efSearch:**Fewer node checks. Fast query times, lower recall.
- ✓ **High efSearch:**Exhaustive local graph searches. High recall, higher latency.

INTERVIEW TIP

Unlike construction parameters (like M), efSearch can be adjusted dynamically at query time to balance latency and recall targets.

What is IVF?

INTERVIEWER EXPECTS: Vector clustering and inverted list space-partitioning logic.

IVF (Inverted File Index) partitions vector spaces into clusters, searching only within the nearest centroids at query time:



INTERVIEW TIP

IVF reduces memory requirements compared to graph-based structures, making it suitable for larger datasets.

What is nlist vs nprobe?

INTERVIEWER EXPECTS: Centroid creation bounds vs query-time explore list depth.

nlist controls partition counts at build time. nprobe defines how many clusters are searched at query time:

- **nlist = 1000:** Build-time clustering splits dataset into 1000 inverted buckets.
- **nprobe = 10:** Evaluates only the 10 closest centroid lists during queries.

MEMORY TRICK

nlist dictates partition density during indexing. nprobe regulates query search widths dynamically.

HNSW vs IVF — how to choose?

INTERVIEWER EXPECTS: Selecting indexes based on accuracy requirements and memory budgets.

Evaluate constraints across query latency targets and memory capacity constraints:

Index Type	Memory Cost	Query Speed	Recall Potential
HNSW	Very High	Extremely Fast	Highest (~98%)
IVF	Moderate	Fast	High (~95%)

SYSTEM DESIGN

Choose HNSW when latency is critical and datasets fit in RAM. Choose IVF when scaling to huge datasets requires clustering.

What is a flat vector index?

INTERVIEWER EXPECTS: Exhaustive search mechanics and baseline utilization.

Flat indexes store vectors without navigation helpers, comparing query parameters against every entry exhaustively:



INTERVIEW TIP

Use flat indexes for small datasets (under 10k vectors) or as a recall baseline. They require no construction time and provide exact matches.

Why benchmark against exact search?

INTERVIEWER EXPECTS: Formulating recall metrics (Recall@K) against exact ground truth.

Without an exact search baseline, you cannot measure the search accuracy loss introduced by ANN approximations:

- ✓ **Exact baseline:** Retrieves 10 mathematically closest vectors.
- ✓ **Recall@10:** Measures what percentage of exact matches are retrieved by the ANN index.

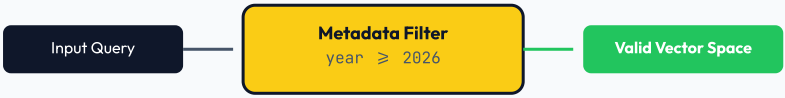
METRIC

If exact search returns `[A B C D E]` and ANN returns `[A B X D Y]`, the search recall score is 60%. Tune parameters to close the gap.

What is metadata filtering?

INTERVIEWER EXPECTS: Distinguishing semantic closeness from tag-based eligibility constraints.

Metadata filtering limits vector candidates to specific scopes (like permissions, location, or date) prior to executing searches:



INTERVIEW TIP

Always enforce pre-filtering (enforcing metadata boundaries before similarity search) to prevent out-of-bounds vectors from entering candidate pools.

Metadata applied incorrectly

INTERVIEWER EXPECTS: Explaining logic failures in filtering pipelines.

If metadata rules are misconfigured, search queries return fast but logically incorrect documents:

- A query match retrieves a highly similar document — from the wrong year.
- LLMs ingest outdated policies, generating incorrect answers.

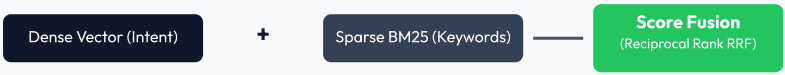
RULE OF THUMB

Similarity scores cannot rescue incorrect metadata filters. Keep metadata schemas versioned and validated in database pipelines.

What is hybrid search?

INTERVIEWER EXPECTS: Combining semantic dense vectors with lexical sparse matching.

Hybrid search combines semantic matches with exact keyword matches, merging score outputs through fusion layers:



INTERVIEW TIP

Use semantic vector search for conceptual intent matches, and sparse keyword search for specific IDs, serial codes, or error codes.

When does lexical search outperform?

INTERVIEWER EXPECTS: Identifying exact string syntax query limits.

Embeddings can lose exact character string alignments. Keyword search performs better when queries contain specific coordinates:

- Product codes (e.g. `SKU_1204`) or API error numbers (e.g. `Error E1042`).
- Legal sections, chemical names, or document version IDs.

PRODUCTION PRACTICE

Do not replace lexical keyword search with embeddings — deploy hybrid search pipelines to cover both exact matches and semantic queries.

How to combine BM25 & vectors?

INTERVIEWER EXPECTS: Normalizing scores or using rank-based Reciprocal Rank Fusion (RRF).

Do not add raw BM25 scores to cosine similarity scores directly (they exist on different scales). Use **RRF**:

Reciprocal Rank Fusion Formula

```
RRF_Score = Σ (1 / (k + rank_m))
```

INTERVIEW TIP

RRF uses relative document rank listings instead of raw score numbers, providing a robust, parameter-free fusion metric.

What is reranking?

INTERVIEWER EXPECTS: Formulating two-stage retrieval architectures.

Initial retrieval retrieves candidate documents quickly. Rerankers (cross-encoders) recalculate scores on candidates for precision:



INTERVIEW TIP

Reranking runs attention calculations over context pairs. Since candidate subsets are small, this process has a minimal latency impact.

Retrieved 10 chunks — 7 are duplicates

INTERVIEWER EXPECTS: Identifying document redundancy issues in vector search.

Standard vector similarity search can retrieve overlapping chunks from the same document. This wastes context tokens without adding new details:

- High similarity scores, but low information diversity.
- LLMs ingest redundant context paragraphs.

MITIGATIONS

Use document-level deduplication, apply diversity algorithms (like MMR), or adjust paragraph chunking rules.

What is MMR?

INTERVIEWER EXPECTS: Formulating relevance-diversity balance logic.

MMR (Maximal Marginal Relevance) selects chunks by balancing relevance to the query with similarity to already retrieved chunks:



INTERVIEW TIP

MMR optimizes information coverage. It penalizes near-duplicate vectors to ensure the model context contains diverse details.

Embeddings excellent — retrieval poor

INTERVIEWER EXPECTS: System debugging across ingestion and query phases.

Do not blame the embedding model immediately. Audit your ingestion pipeline first:

- **Chunk sizes:**Chunks may be too large (adds noise) or too small (lacks context).
- **Query patterns:**Queries might require formatting alignment or translation.
- **Filters:**Incorrect metadata boundaries could be excluding results.

INTERVIEW TIP

Retrieval quality is dictated by document preprocessing and chunking structures, not just embedding vector matching.

Changing the embedding model

INTERVIEWER EXPECTS: Impact on vector spaces and indexing pipelines.

Changing models modifies the coordinate space. You cannot mix old and new vectors — you must re-index the database:



INTERVIEW TIP

Treat model updates as breaking database migrations. Run offline index regeneration and versioning before redirecting traffic.

Why version embedding models?

INTERVIEWER EXPECTS: Tracking schema dependencies in vector spaces.

Because the embedding function establishes the vector coordinate geometry. Store model metadata schemas next to indices:

- `"embedding_model": "model-v2"`
- `"embedding_version": "2026-01"`

PRODUCTION RULE

Treat embeddings like database schemas. Store version tags in metadata to prevent database corruption during updates.

No results after deployment

INTERVIEWER EXPECTS: Formulating systematic diagnostic debugging steps.

When database query results fail, isolate variables sequentially:



INTERVIEW TIP

Start with pipeline observability dashboard metrics (dimension checks, filter constraints) rather than guessing index parameters.

How to estimate vector storage?

INTERVIEWER EXPECTS: Memory math calculations factoring index overhead.

Compute memory usage for raw vectors first, then factor in metadata and index graph overheads:

Storage = N × D × 4 bytes

```
(+ HNSW graph link connection overhead up to 100%)
```

INTERVIEW TIP

Never calculate raw vector sizes alone — note that database IDs, metadata fields, and graph index structures add significant memory overhead.

What is vector quantization?

INTERVIEWER EXPECTS: Knowledge of vector memory compression techniques.

Quantization compresses float32 values into lower-precision representation layers to save memory:



INTERVIEW TIP

Quantization saves memory but introduces a minor loss in vector precision. Assess the accuracy impact before deploying to production.

100M vectors — strict latency requirements

INTERVIEWER EXPECTS: Scaling strategies (sharding, compression, indexes).

Do not simply suggest "use HNSW". Scale requires combining partitioning, quantization, and sharding:

- Partition the index using IVF to divide the search space.
- Compress vectors using scalar quantization to fit them in memory.
- Shard the database across search nodes to query indices in parallel.

INTERVIEW TIP

At large scale, design a two-stage retrieval pipeline: perform fast, compressed candidate searches, and then rerank the top candidates.

Similar leaderboard scores — how to choose?

INTERVIEWER EXPECTS: Domain-specific validation over public scores.

Public leaderboards do not represent domain-specific retrieval quality. Build a test set to evaluate models on your actual workload:

Workload Validation Metrics

Recall@K • Precision@K • Latency • Host Costs

INTERVIEW TIP

The best embedding model is the one that performs best for your domain data, under your latency and cost constraints.

Design a production-grade retrieval system

INTERVIEWER EXPECTS: Connecting every retrieval component into an enterprise pipeline.

Avoid treating the vector database as a black box. Design retrieval as an end-to-end pipeline:



TAKEAWAY

Connect pre-processing, indexing, sharding, hybrid fusion, reranking, and evaluation into a cohesive, measurable system.

INDEX CHEAT SHEET

Solutions

Vector DB Cheat Sheet

01. Embeddings Learned continuous vector space representations.

02. Similarity Cosine → orientation; Euclidean → distance.

03. Exact vs ANN Exact is precise; ANN scales using sub-linear logic.

04. HNSW Connectivity M controls links; efSearch controls query exploration.

05. IVF Clustering nlist defines clusters; nprobe defines clusters searched.

06. Filtering Enforce metadata checks pre-search to protect bounds.

07. Hybrid Search Combine dense intent with sparse BM25 keywords.

08. Quantization Compresses float32 representations to save memory.

09 & 10. Reranking & System Versioning Rerank candidates to improve precision; treat embedding version updates as breaking schema changes.

Design your retrieval system as a pipeline, not a black box index.

Devadharshani A Follow Topmate

Repost

INTERACTIVE OUTRO

Which index concept do you struggle with?

1 — Embeddings & coordinate dimensions
2 — Cosine vs Euclidean similarity math
3 — HNSW layer graph traversal parameters
4 — IVF centroid nlist/nprobe boundaries
5 — Hybrid BM25 Reciprocal Rank Fusion
6 — Production scaling constraints (100M+ vectors)
Comment your choice below!

"A vector database does not magically provide correct answers. It only searches representation layers."

Day 5 Complete
Save this series
Next: 40 LLM Interview Questions (Transformers & Attention)

Devadharshani A Follow Topmate Repost